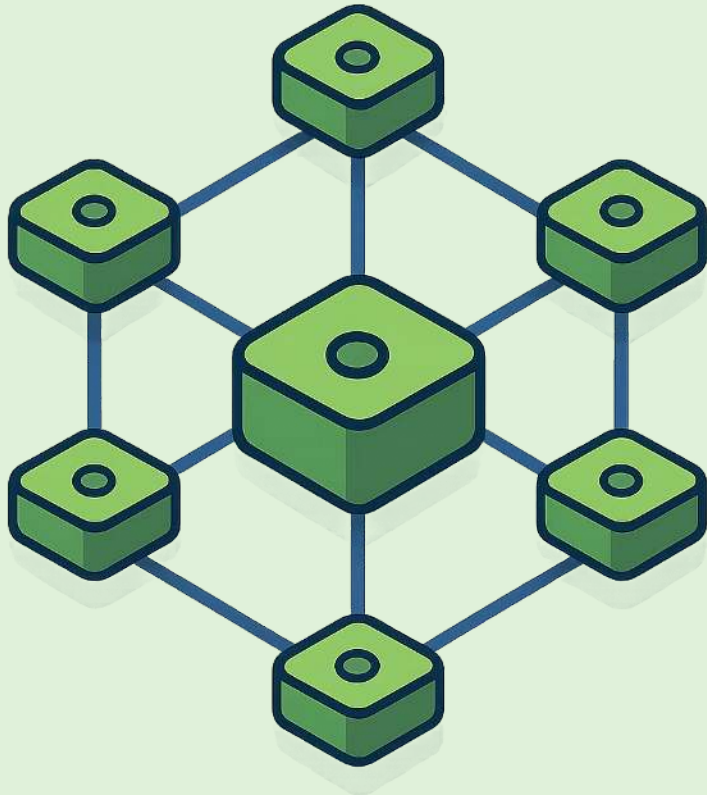




Microservices

Einführung

Unterrichtseinheit 1/4



Tobias Delp

39 Jahre

Berufliche Erfahrung

Seit einem Jahr Dozent in der Erwachsenenbildung

Mehr als 10 Jahre Erfahrung als Fullstack-Entwickler

2 Jahre als Entwicklungsleiter

3 Jahre als DevOps-Engineer

Hobbies



BOULDERN



JAPAN



Überblick der Unterrichtseinheit

01

Inhalte

Monolith, Microservices, Prinzipien,
Vor- & Nachteile

02

Ablauf

Einstieg → Erarbeitung →
Vertiefung → Reflexion →
Abschluss

03

Erwartungen

Aktive Mitarbeit, Fragen stellen, eigene Ideen einbringen

Lernziele

Nach dieser Einheit kannst du folgende Kompetenzen nachweisen:

1

Erklären

Du kannst beschreiben, was Microservices sind und wie sie funktionieren.

2

Unterscheiden

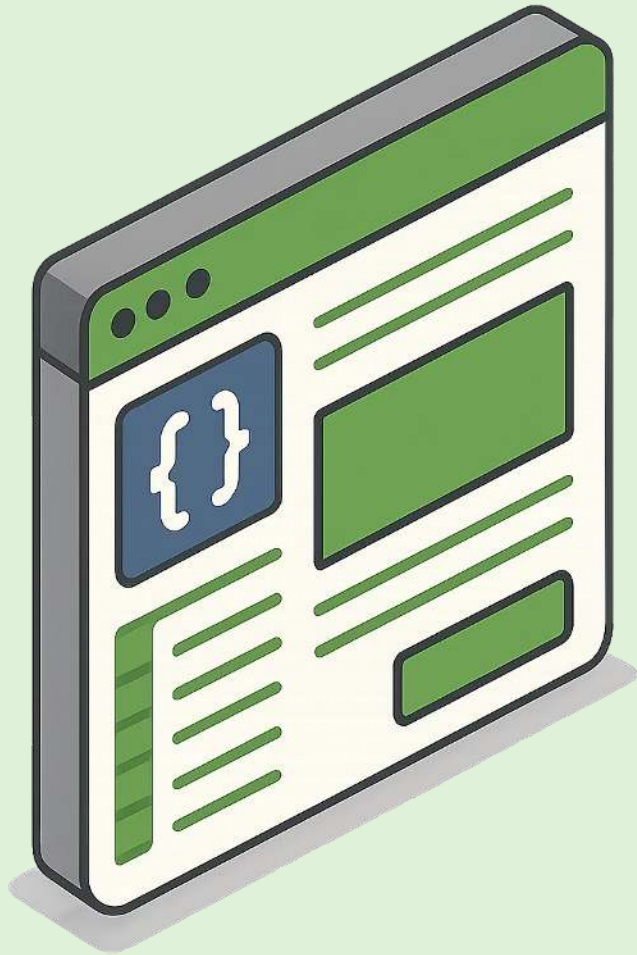
Du erkennst die wesentlichen Unterschiede zwischen Monolith und Microservices.

3

Bewerten

Du kannst Vorteile und Herausforderungen benennen und kritisch einordnen.

Einstieg: Leitfragen



Welche großen Webanwendungen kennt ihr?

Denke an Plattformen, die du täglich nutzt – von Social Media bis Online-Shopping.

Wie könnten diese technisch aufgebaut sein?

Läuft alles auf einem Server? Gibt es viele verschiedene Komponenten?

Welchen Mehrwert erwarten wir?

Was muss ein modernes System leisten – Geschwindigkeit, Zuverlässigkeit, Skalierung?



 5 MINUTEN

Denkphase

Nimm dir einen Moment und überlege für dich:

- Welche Apps oder Webseiten nutzt du regelmäßig?
- Was passiert wohl im Hintergrund, wenn du auf einen Button klickst?
- Wie viele Systeme könnten dabei zusammenarbeiten?

 **Notiere deine Gedanken**

Deine Antwort ist gefragt!

Aufgabe 1

Schreibe **2–3 Webanwendungen**, die du kennst, in den Chat.

Aufgabe 2 (optional)

Füge eine erste **Idee zum technischen Aufbau** hinzu – wie glaubst du, ist die App gebaut?

 Keine falschen Antworten – es geht um deine erste Einschätzung!

Beispiel-Anwendungen im Überblick



Social Media

Instagram, TikTok, Twitter/X

→ Milliarden Nutzer, Echtzeit-Feeds, Kommentare, Stories



Streaming

Netflix, YouTube, Spotify

→ parallele Streams, personalisierte Empfehlungen, hohe Last



E-Commerce

Amazon, Zalando, eBay

→ Produktsuche, Bestellungen, Zahlungen, Lieferverfolgung



Kommunikation

WhatsApp, Slack, Teams

→ Nachrichten, Sprach- & Videoanrufe, Dateiübertragung



 VERTIEFUNGSRAGE

Was passiert im Hintergrund?

Du klickst bei Amazon auf „Jetzt kaufen“ – was läuft technisch in diesem Moment ab?

- Wer prüft, ob du eingeloggt bist?
- Wie wird die Zahlung verarbeitet?
- Wer aktualisiert den Lagerbestand?
- Wie wird die Bestellbestätigung verschickt?
- Wer löst den Versandprozess aus?

 Jede dieser Fragen steckt hinter einem eigenen System – oder sogar einem eigenen Service ...

Was steckt dahinter? – Mögliche Antworten

Hinter dem „Jetzt kaufen“-Button verbergen sich mehrere unabhängige Systeme:

Authentifizierung

Prüft, ob der Nutzer eingeloggt ist und die Session gültig ist

Zahlungsservice

Verarbeitet die Zahlung über Kreditkarte, PayPal o. ä.

Lagerservice

Prüft die Verfügbarkeit und reduziert den Lagerbestand

Benachrichtigungsservice

Versickt die Bestellbestätigung per E-Mail

Versandservice

Löst den Lieferprozess aus und erstellt das Versandetikett

Anforderungen moderner Systeme

Skalieren

Bei hoher Last müssen Systeme flexibel wachsen – z. B. bei Verkaufsspitzen am Black Friday.



Hochverfügbar

Ausfälle sind nicht tolerierbar – Nutzer erwarten 24/7-Betrieb ohne Unterbrechungen.

Schnell entwickeln

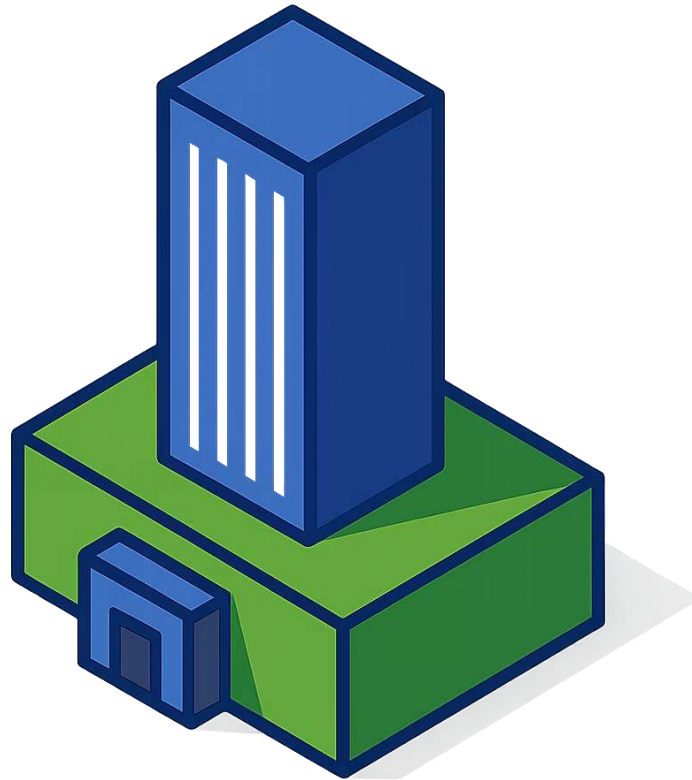
Neue Features müssen zügig bereitgestellt werden – der Markt wartet nicht.



Kann ein einzelnes System das wirklich leisten?

Skalierung, Hochverfügbarkeit und schnelle Entwicklung – gleichzeitig, in einer einzigen Anwendung?

 Überlege: Was passiert, wenn dieses eine System ausfällt oder zu langsam wird?



ARCHITEKTUR

Der Monolith

Eine Anwendung = ein großes, zusammenhängendes System

Alle Funktionen – Benutzeroberfläche, Geschäftslogik, Datenbankzugriff – sind in **einer einzigen Codebasis** vereint und werden gemeinsam deployed.

Viele klassische Unternehmensanwendungen wurden so gebaut – einfach zu starten, aber schwer zu warten.

Eigenschaften eines Monolithen

Eine Codebasis

Sämtlicher Code – Frontend, Backend, Datenbanklogik – lebt in einem einzigen Repository und wird als Einheit verwaltet.

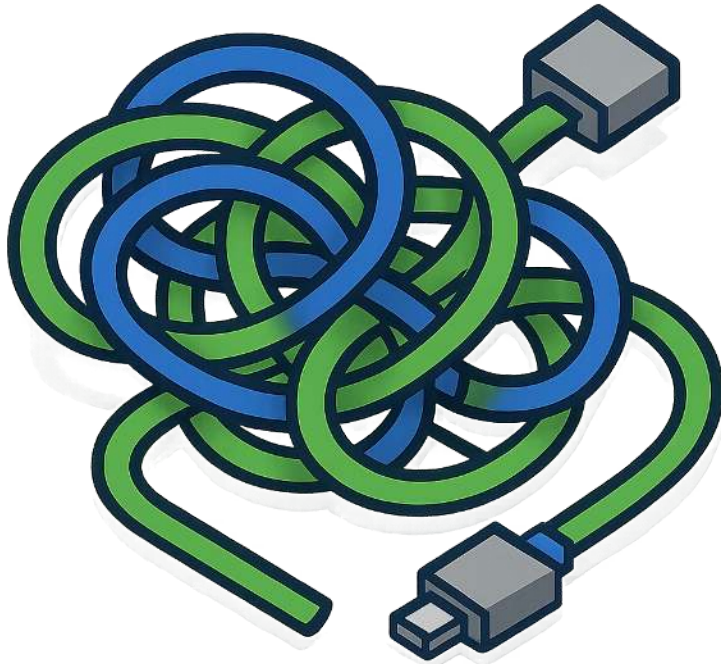
Stark gekoppelt

Komponenten sind eng miteinander verknüpft. Eine Änderung an einer Stelle kann unerwartete Auswirkungen auf andere Teile haben.

Gemeinsames Deployment

Jede Änderung – egal wie klein – erfordert ein vollständiges Neu-Deployment der gesamten Anwendung.

Welche Probleme könnten entstehen?



Stell dir vor, du arbeitest in einem Team von 50 Entwickler:innen an einer einzigen, riesigen Codebasis.

- Was passiert, wenn jemand einen Fehler einbaut?
- Wie lange dauert das Deployment?
- Wie skalierst du nur einen Teil des Systems?

 Denke darüber nach und notiere deine Ideen – gleich teilen wir sie!

Probleme des Monolithen – deine Ideen

Schreibe deine Vermutungen in den Chat. Was könnte in einem monolithischen System zum Problem werden?

Denkanstoß 1

Was passiert, wenn du nur die Zahlungslogik skalieren möchtest, aber der Rest nicht nötig ist?

Denkanstoß 2

Ein kleiner Bug-Fix erfordert das Deployment der gesamten Anwendung – was bedeutet das für den Betrieb?

Denkanstoß 3

Wie schwer ist es, neue Technologien einzuführen, wenn alles eng verknüpft ist?

Nachteile des Monolithen

1

Schwer skalierbar

Nur das gesamte System kann hochskaliert werden – einzelne Komponenten lassen sich nicht gezielt verstärken.

2

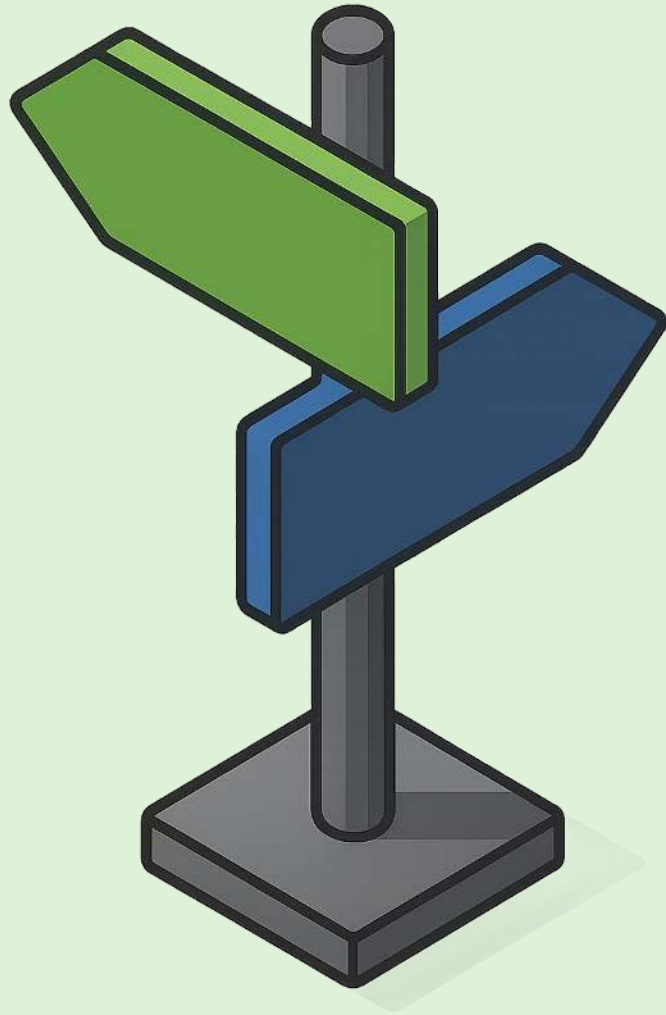
Riskante Änderungen

Jede Änderung betrifft potenziell das ganze System. Bugs können sich unbemerkt ausbreiten.

3

Lange Deployments

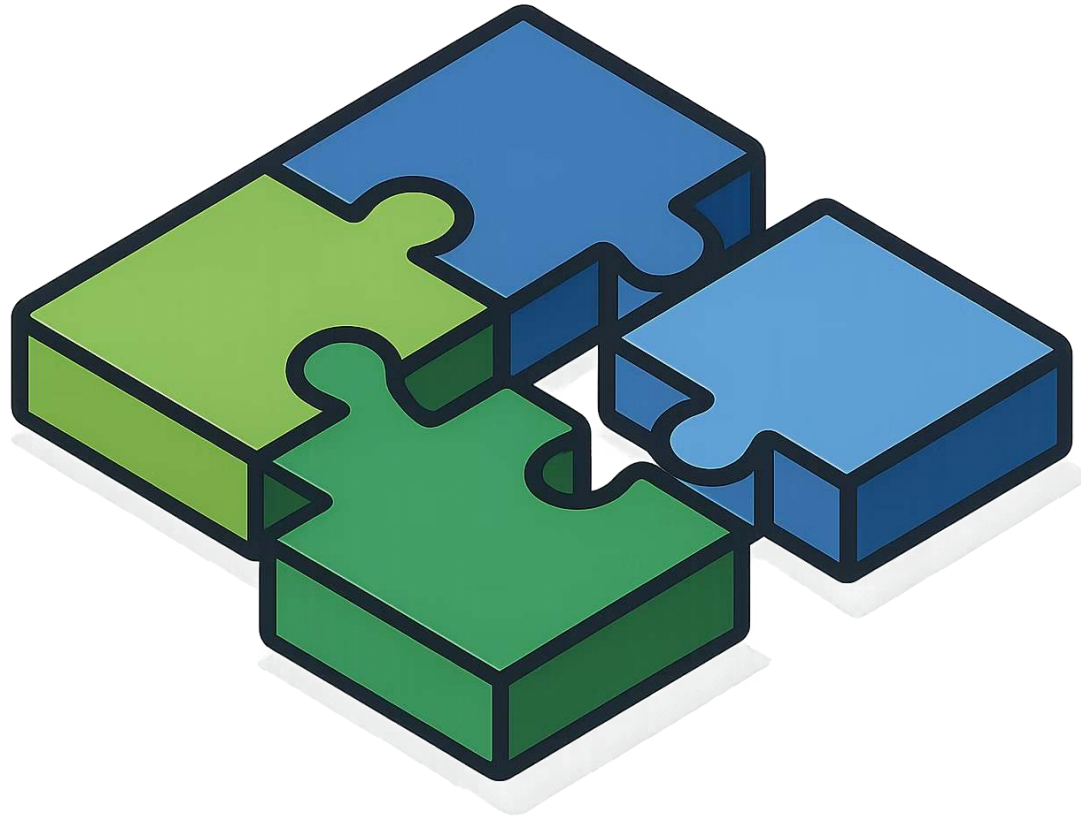
Das gesamte System muss neu gebaut und ausgerollt werden – auch für kleinste Korrekturen.



**Der entscheidende
Wendepunkt**

**Komplexe
Anforderungen
treffen auf starre
Systeme**

Die Lösung: ein grundlegend anderes Architekturprinzip – **Microservices**.



DEFINITION

Was sind Microservices?

Eine Anwendung besteht aus vielen kleinen, unabhängigen Services – jeder mit einer klaren Aufgabe.

Anstatt alles in einer Codebasis zu bündeln, wird die Anwendung in **eigenständige, lose gekoppelte Dienste** aufgeteilt, die unabhängig voneinander entwickelt, deployed und skaliert werden können.

Die Grundidee von Microservices



Jeder Service hat eine Aufgabe

Das **Single Responsibility Principle**: Ein Service macht genau eine Sache – und das richtig. Z. B. nur Authentifizierung oder nur Bestellverwaltung.



Kommunikation über APIs

Services sprechen miteinander über klar definierte Schnittstellen – meist **REST-APIs** oder Messaging-Systeme. Interna bleiben verborgen.

Beispiel: Microservices im Online-Shop

Ein typischer Online-Shop lässt sich in klar getrennte, unabhängige Services aufteilen:



User Service

Registrierung, Login,
Profilverwaltung &
Authentifizierung



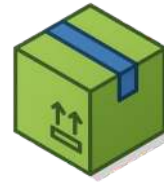
Product Service

Produktliste, Suche,
Kategorien &
Bestandsverwaltung



Payment Service

Zahlungsabwicklung,
Rechnungen &
Transaktionsstatus



Order Service

Bestellungen anlegen,
verfolgen & Statusupdates
versenden

Welche weiteren Services wären sinnvoll?

Der Online-Shop hat bereits vier Services. Aber es fehlen noch einige wichtige Funktionen ...

Denkanstoß

- Wie werden Bewertungen verwaltet?
- Wer kümmert sich um Benachrichtigungen (E-Mail, Push)?
- Gibt es einen eigenen Such-Service?

Deine Aufgabe

Schreibe deine Ideen in den Chat! Nenne den Service und seine Aufgabe – z. B. „*Review Service – verwaltet Kundenbewertungen*“.

Designprinzipien: Teil 1



Lose Kopplung

Services kennen sich nur über ihre APIs. Interne Änderungen eines Services beeinflussen andere **nicht**. Teams können unabhängig arbeiten.

Hohe Kohäsion

Zusammengehörige Logik bleibt zusammen. Jeder Service ist in sich geschlossen und hat eine klare, fokussierte Verantwortung.

- ❏ Diese zwei Prinzipien sind das Fundament guter Microservice-Architektur.

Designprinzipien: Teil 2



Unabhängige Deployments

Jeder Service kann separat gebaut, getestet und in Produktion gebracht werden – **ohne andere Services zu stoppen**. Kleinere Releases, geringeres Risiko.



Technologieunabhängigkeit

Verschiedene Services können in verschiedenen Programmiersprachen oder Frameworks geschrieben sein – das **beste Werkzeug für jede Aufgabe**.

Vorteile: Teil 1

Skalierbarkeit

Nur der Service, der unter Last steht, muss skaliert werden. Beim Black Friday wird der Checkout-Service hochskaliert – nicht das gesamte System.

Flexibilität

Teams können Technologien, Frameworks und Datenbanken frei wählen – je nachdem, was für ihren Service am besten geeignet ist.



Vorteile: Teil 2

Schnellere Entwicklung

Kleine, überschaubare Services sind leichter zu verstehen, zu testen und zu deployen. Features gelangen schneller in die Produktion.

Unabhängige Teams

Jedes Team besitzt seinen Service vollständig – von der Entwicklung bis zum Betrieb. Weniger Koordinationsaufwand, mehr Eigenverantwortung.

Welcher Vorteil zählt für euch am meisten?



Skalierbarkeit



Flexibilität



Schnelligkeit



Teamautonomie

 Schreibe deine Wahl in den Chat – und begründe kurz, warum!

Herausforderungen: Teil 1

Komplexität

Viele Services bedeuten viele bewegliche Teile. Deployment, Konfiguration und Orchestrierung werden deutlich aufwendiger als beim Monolithen.

Netzwerkabhängigkeit

Services kommunizieren über das Netzwerk – das bringt Latenz, Timeouts und potenzielle Ausfälle mit sich, die behandelt werden müssen.



Herausforderungen: Teil 2



Monitoring

Fehler müssen über viele Services hinweg nachverfolgt werden. Ohne gutes Logging und Tracing ist die Fehlersuche extrem schwierig.



Datenkonsistenz

Jeder Service hat idealerweise seine eigene Datenbank. Daten konsistent zu halten und Transaktionen über mehrere Services zu koordinieren ist komplex.



Herausforderungen: Teil 3



Lokales Development

Die lokale Entwicklung und das Testen erfordern oft, dass viele Microservices gleichzeitig gestartet und konfiguriert werden, was ressourcenintensiv und komplex sein kann.



Breaking Changes

Inkompatible Änderungen in der API eines Services können alle abhängigen Services betreffen. Dies erfordert strenge Versionskontrolle und sorgfältige Abstimmung.

 10 MINUTEN

 DISKUSSION

Welche Herausforderung erscheint euch kritisch?

Schreibe deine Antwort in den Chat – welche Herausforderung würde dich am meisten beschäftigen und warum?

Im Anschluss tauschen wir uns kurz aus und diskutieren gemeinsam mögliche Lösungsansätze.

 Gibt es Strategien, um diese Herausforderung zu meistern?

Zusammenfassung

Monolith

Eine Codebasis, stark gekoppelt, gemeinsames Deployment

Einfach zu starten, aber schwer zu skalieren und zu warten

Riskante Änderungen, lange Release-Zyklen

Microservices

Viele kleine, unabhängige Services mit klarer Aufgabe

Skalierbar, flexibel, schnellere Entwicklung & Teamautonomie

Höhere Komplexität, Netzwerk- und Datenkonsistenz-Herausforderungen

✅ Beides hat seinen Platz – die Wahl hängt von Anforderungen, Teamgröße und Kontext ab.

Wo würdet ihr Microservices einsetzen?

Denke an ein konkretes Projekt oder Szenario – reales oder fiktives. Wann macht Microservice-Architektur Sinn?

Wann spricht vieles dafür?

- Großes Team, viele parallele Features
- Unterschiedliche Last auf verschiedene Teile
- Schnelle, unabhängige Releases nötig

Wann lieber Monolith?

- Kleines Team, einfache Anwendung
- Schneller Prototyp gefragt
- Kein Ops-Team verfügbar

 Schreibe dein Beispiel in den Chat!

Abschluss & Ausblick

Gut gemacht! Du hast heute die Grundlagen moderner Softwarearchitektur kennengelernt. In den nächsten Einheiten bauen wir darauf auf:

01

REST APIs

Wie kommunizieren Services miteinander? Aufbau, HTTP-Methoden, JSON-Datenformat

02

Kommunikation zwischen Services

Synchrone vs. asynchrone Kommunikation, Message Queues, Event-driven Architecture

03

Docker

Wie werden Microservices verpackt und betrieben? Container, Images, Docker Compose

